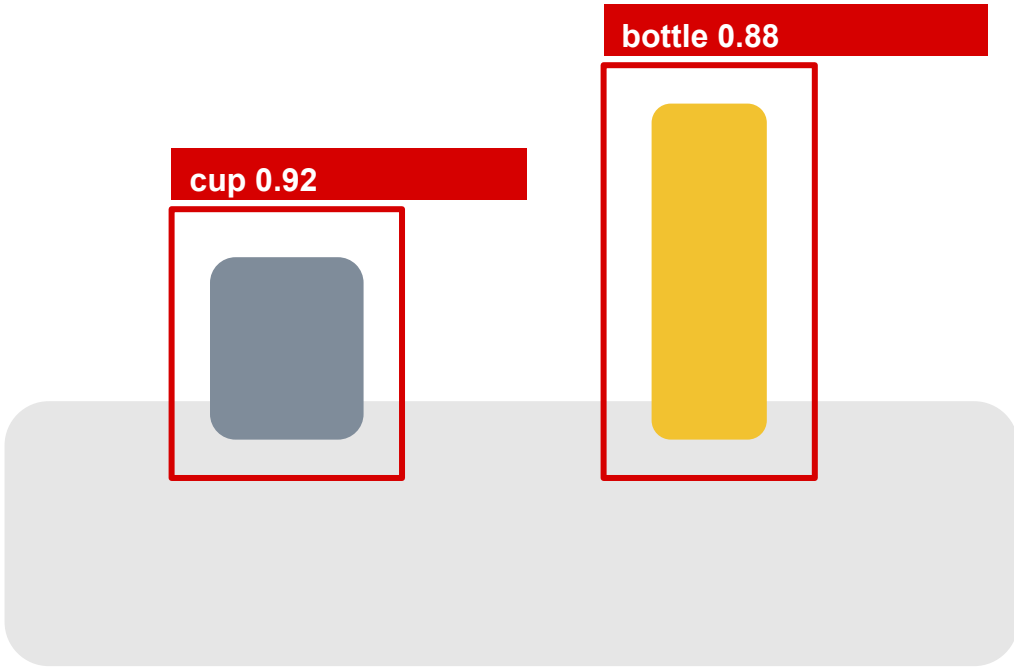
ROBOTIC ARM COURSE · LESSON 9 · OBJECT DETECTION

AI Object Detection

Finding objects in a camera image with a pretrained model, one box per object

Day 1 · Bounding boxes and a pretrained detector

Day 2 · Cleaning up detections and pointing the arm



RECAP · LESSONS 6 TO 8

Where Lessons 6 to 8 Left Off

Inverse kinematics turns a target into joint angles. But the arm still cannot find a cup on the table.

QUESTION 1

What is it?

a label, such as cup

+

QUESTION 2

Where is it?

a position in the image

=

ANSWER

A box

label, position and score

GIVEN IN LESSON 6

The target **(301.78, 301.78, 176.78)** mm needs θ_B = **45°**, θ_S = **45°**, θ_E = **−45°**.

TODAY

A detector finds every object in a camera image and draws a box around it. Then we can point the arm at it.

WHY DETECTION

From Camera to Base Angle

Five steps take a photo to a turn of the base. Each step is a few lines of code.

1 · IMAGE
Photo
from the arm camera

→

2 · DETECTOR
Boxes
box, label, score

→

3 · CLEAN-UP
Filter
threshold and NMS

→

4 · TARGET
Pick one
best box of a class

→

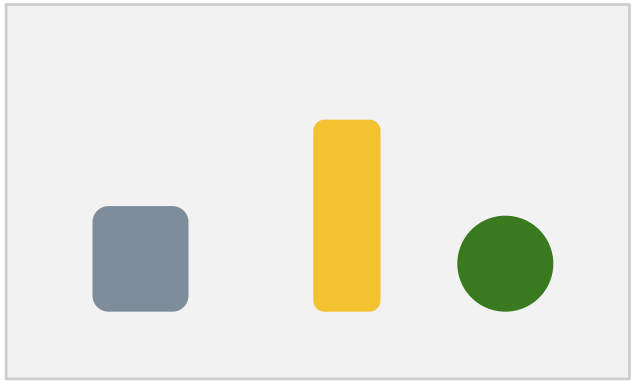
5 · ANGLE
Turn
which way for the base

DAY 1
Steps 1 and 2: what a box is, and how to run a pretrained detector

DAY 2
Steps 3, 4 and 5: clean up the boxes, pick a target and point the arm

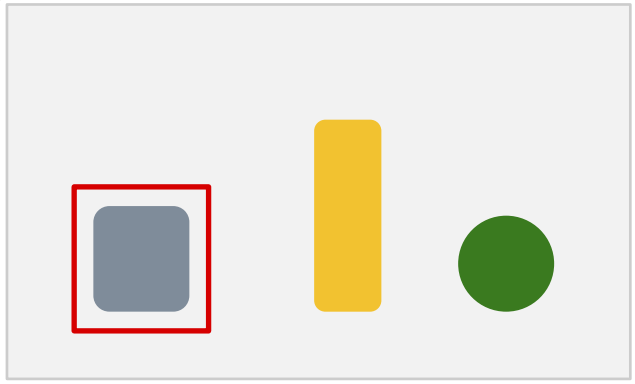
DAY 1 · WHAT DETECTION IS

Classification, Localization, Detection



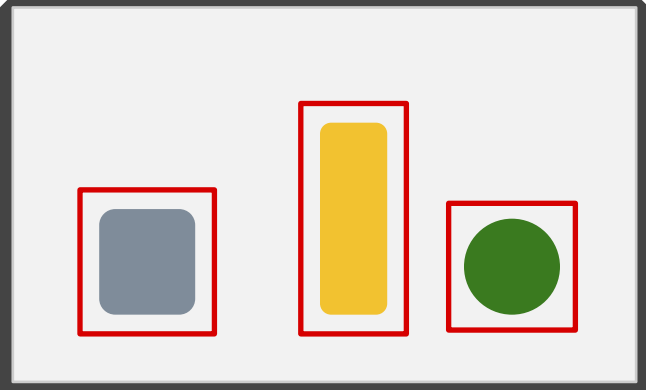
CLASSIFICATION

One label for the whole image: **“cup”**. It says what, not where.



LOCALIZATION

One box around one object. It says where.



DETECTION

A box, a label and a score for every object in the image.

DAY 1 · IMAGE COORDINATES

Pixels Have Their Own Axes

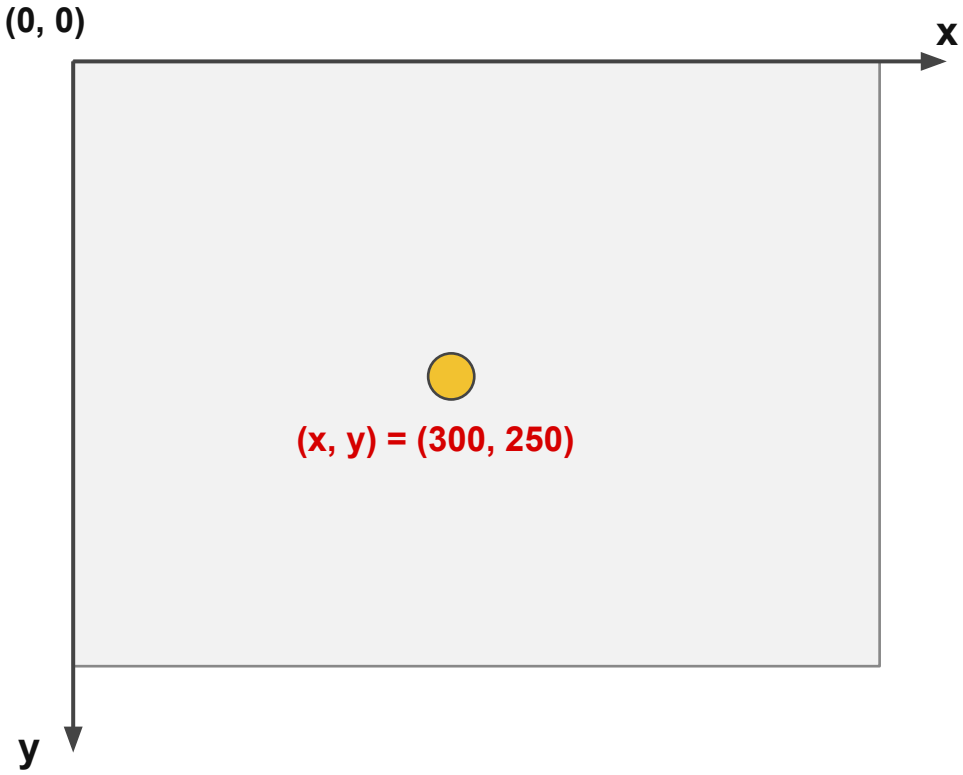
Origin: (0, 0) is the top-left corner

x grows to the right

y grows **downward**

ONE CHANGE OF HABIT

In the arm lessons, up was y or z. In an image, a larger y means lower in the picture.



A 640 × 480 image

DAY 1 · BOUNDING BOXES

Two Ways to Write a Box

CORNER FORMAT

(x_min, y_min, x_max, y_max)

(120, 80, 360, 320)

WIDTH AND HEIGHT FORMAT

(x, y, width, height)

(120, 80, 240, 240)

The two formats hold the same information. The detector in this lesson uses corner format.

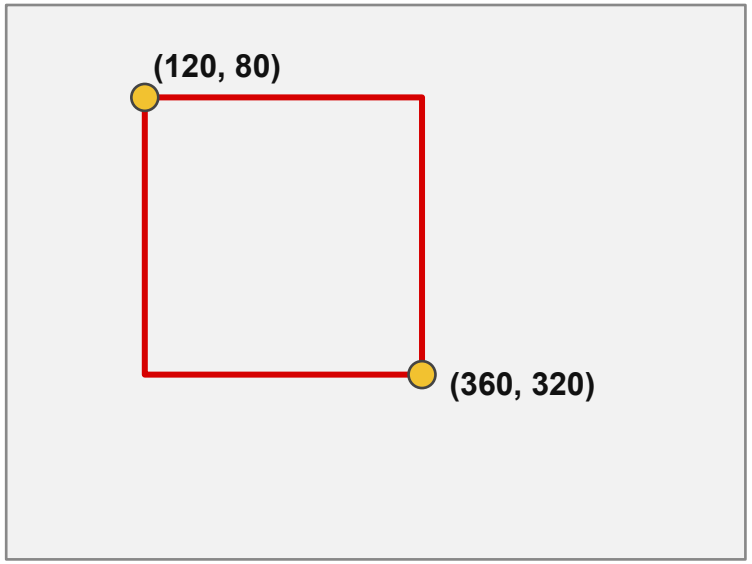


Image coordinates: top-left origin, y down

DAY 1 · BOUNDING BOXES

Worked Example: Box Measurements

Measure the box **(120, 80, 360, 320)** from its corners.

WIDTH

x_max − x_min = 360 − 120

240

HEIGHT

y_max − y_min = 320 − 80

240

CENTER

((120 + 360) ÷ 2, (80 + 320) ÷ 2)

(240, 200)

AREA

width × height = 240 × 240

57,600 px²

So (120, 80, 240, 240) in width and height format is the same box.

DAY 1 · TRY IT YOURSELF

Try It Yourself #1

Find the width, height, center and area of the box, then write it as (x, y, width, height).

(50, 60, 250, 160)



Drawn at the same scale as the pixels

SOLUTION

Width 250 − 50 = 200, height 160 − 60 = 100

Center (150, 110)

Area 20,000 px²

(50, 60, 200, 100)

DAY 1 · OVERLAP

Intersection over Union

To compare a model's box with a true box we need one number for how much they overlap.

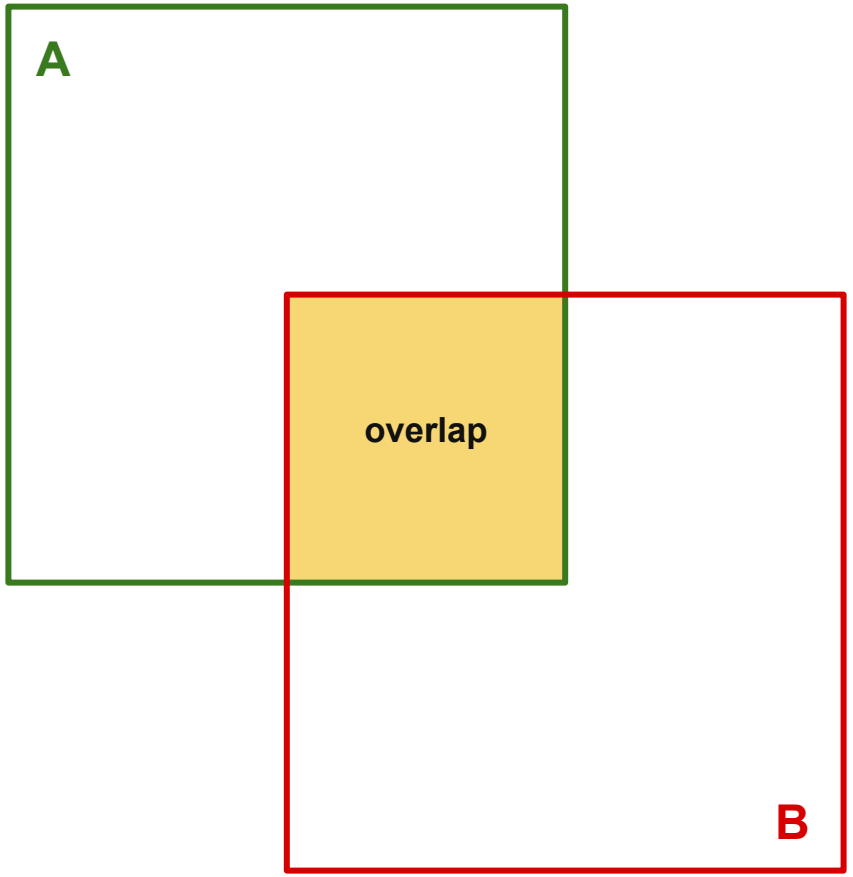
IOU

overlap area ÷ combined area

0 means the boxes do not touch

1 means the boxes are identical

0.5 or more usually counts as a match



DAY 1 · OVERLAP

Worked Example: IoU

A = **(100, 100, 300, 300)** and B = **(200, 200, 400, 400)**

STEP 1 · OVERLAP

Larger mins, smaller maxes:
(200, 200, 300, 300)

100 × 100 = 10,000

STEP 2 · COMBINED AREA

Each box is 200 × 200 = 40,000

40,000 + 40,000 − 10,000 = 70,000

STEP 3 · IoU

overlap ÷ combined area

10,000 ÷ 70,000 = 0.14

COMMON MISTAKE

Adding the two areas and forgetting to subtract the overlap counts the overlap twice.

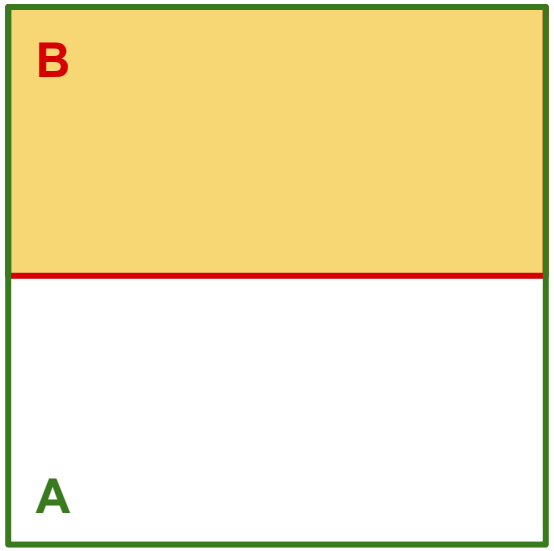
0.14 is well under 0.5, so these two boxes would not count as a match.

DAY 1 · TRY IT YOURSELF

Try It Yourself #2

Find the IoU of these two boxes.

A = (0, 0, 100, 100)
B = (0, 0, 100, 50)



SOLUTION

B sits completely inside A, so the overlap is B's area, 5,000. The combined area is A's area, 10,000.

IoU = 0.50

Bonus: what is the IoU of two identical boxes? It is 1.

DAY 1 · PRETRAINED MODELS

Someone Already Taught It

INPUT

An image

one camera photo

→

PRETRAINED MODEL

Already learned

from hundreds of thousands of labeled photos

→

OUTPUT

A list

box, label and score for each object

WHAT IT KNOWS

COCO's **80 classes**: cup, bottle, scissors, apple and so on. Nothing else.

WHAT TO WATCH FOR

Its training photos were not taken by our arm camera, so some objects are harder for it.

EXAMPLE OUTPUT

```
cup      0.92   (120, 80, 360, 320)
bottle   0.88   (450, 150, 600, 470)
```

DAY 1 · CHECK WITH CODE

Check It with Code

PYTHON

```python
from detect_helper import run_detector, draw_detections

dets = run_detector("table.png")
for d in dets:
    print(d["label"], round(d["score"], 2), d["box"])
draw_detections("table.png", dets, "detections.png")
```

WHAT EACH PART DOES

run_detector runs the pretrained model. draw_detections saves a picture with the boxes drawn on it.

YOUR JOB

Run it on three arm-camera photos. Find one mistake: a wrong label, a missed object or a box on nothing.

DAY 1 · RECAP

Day 1 Recap

BOX

(x_min, y_min, x_max, y_max)

Top-left origin, y grows down.

IOU

overlap ÷ combined

0 means apart, 1 means identical.

PRETRAINED MODEL

Knows 80 classes

Already trained, so we only run it.

OUTPUT

Box + label + score

One item for every object it finds.

Next time: the model's output is messy. We clean it up, then use it to point the arm at a target.

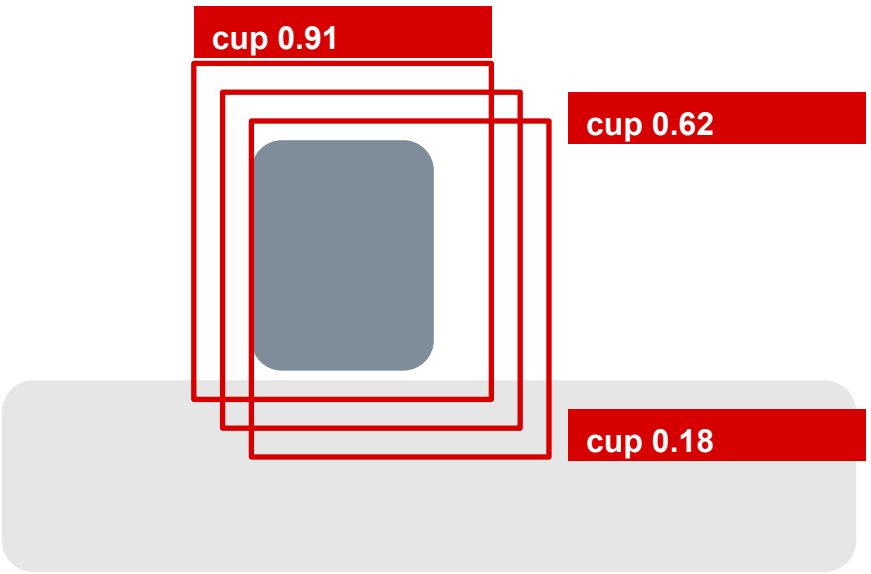
DAY 2

Cleaning Up and Pointing the Arm

Day 1 gave us boxes. Now we keep the good ones, remove the duplicates and use the best box to turn the base.

DAY 1 IN ONE LINE

A detector returns a box, a label and a score for each object

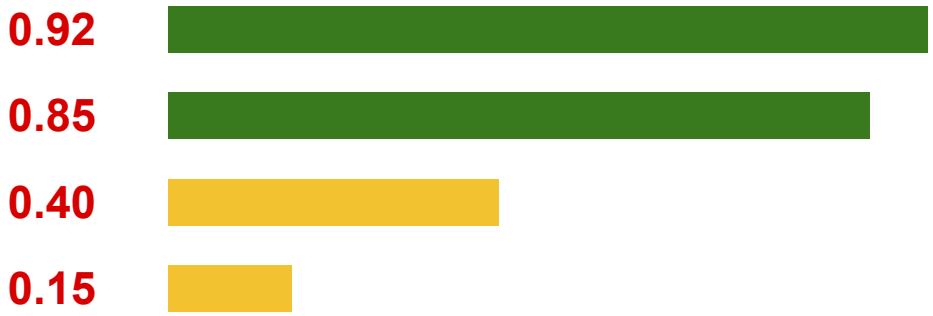


DAY 2 · CONFIDENCE

How Sure Is the Model?

Every detection has a score from 0 to 1. We choose a threshold and throw away anything below it.

FOUR DETECTIONS



Threshold **0.9** — **keeps 1**

Threshold **0.5** — **keeps 2**

Threshold **0.3** — **keeps 3**

THRESHOLD TOO LOW

False alarms: boxes on nothing

THRESHOLD TOO HIGH

Missed objects that are really there

DAY 2 · LAB

Lab: Threshold Sweep

1. Run the detector on one photo with conf = 0.10, 0.25, 0.50 and 0.80.

2. Count the boxes each setting gives, and how many are right.

3. Fill in the table.

conf	boxes found	boxes right
0.10	—	—
0.25	—	—
0.50	—	—
0.80	—	—

THEN DECIDE

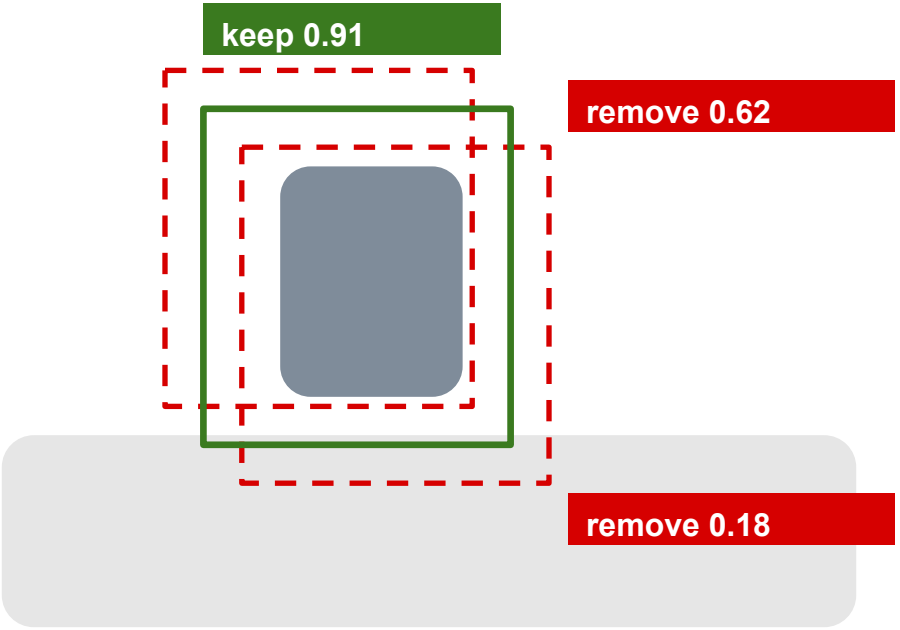
Which setting would you trust to guide a robot? Defend it in one sentence.

DAY 2 · DUPLICATES

Too Many Boxes

A detector often draws several boxes on one object.
Non-maximum suppression (NMS) keeps the best one.

1. Sort the boxes by score
2. Keep the best box
3. Remove boxes with the same label that overlap it by more than **0.5 IoU**
4. Repeat with the next box left



YOLO already does this inside the model. We do it by hand to see how it works.

DAY 2 · WORKED EXAMPLE

NMS Example

Three cup detections. Overlap threshold is 0.5.

Box	(x_min, y_min, x_max, y_max)	Score
A	(100, 100, 300, 300)	0.90
B	(120, 110, 320, 310)	0.70
C	(500, 200, 600, 300)	0.80

SOLUTION

A has the top score, so keep it. B vs A: overlap = 180 × 190 = 34,200; union = 40,000 + 40,000 − 34,200 = 45,800.

IoU(A, B) = 0.75 → remove B

C does not touch A, so IoU = 0 and C stays.

Kept: A and C

DAY 2 · TRY IT YOURSELF

Try It Yourself #3

Run NMS with threshold 0.5. Which boxes are kept?

Box	(x_min, y_min, x_max, y_max)	Score
A	(0, 0, 100, 100)	0.90
B	(10, 0, 110, 100)	0.80
C	(300, 300, 400, 400)	0.70

SOLUTION

Keep A. B vs A: overlap = 90 × 100 = 9,000; union = 10,000 + 10,000 − 9,000 = 11,000.

IoU(A, B) = 0.82 → remove B

C is far away → keep. Kept: A and C

DAY 2 · PUTTING IT TOGETHER

Clean the List, Then Count

STEP 1 **Raw detections** → STEP 2 **Drop low scores** → STEP 3 **Run NMS** → STEP 4 **Count**

Label	Score	What happens
cup	0.91	kept
cup	0.88	kept (a different cup)
cup	0.62	removed by NMS (same cup as 0.91)
cup	0.30	dropped (below 0.5)
bottle	0.80	kept

Result: 2 cups, 1 bottle

DAY 2 · SCORING THE DETECTOR

Precision and Recall

A detection counts as correct when its box matches a real object (IoU of 0.5 or more) with the right label.

TP
Correct detection

FP
Box on nothing, or wrong label

FN
Real object the model missed

PRECISION
TP ÷ (TP + FP)
Of the boxes drawn, how many were right?

RECALL
TP ÷ (TP + FN)
Of the real objects, how many did it find?

Example: 8 boxes drawn, 6 correct, 10 real objects. Precision = 6 ÷ 8 = **0.75**. Recall = 6 ÷ 10 = **0.60**.

DAY 2 · TRY IT YOURSELF

Try It Yourself #4

The detector draws **5 boxes**. **4** are correct. The photo really contains **8 objects**.

Find the precision and the recall.

SOLUTION

TP = 4, FP = 5 − 4 = 1, FN = 8 − 4 = 4.

Precision = 4 ÷ 5 = 0.80

Recall = 4 ÷ 8 = 0.50

Mostly right when it speaks, but it misses half the objects.

DAY 2 · POINTING THE ARM

From Box to Turn Angle

1 · BOX CENTER

$$cx = (x_min + x_max) \div 2$$

2 · OFFSET FROM IMAGE CENTER

$$dx = cx - W \div 2$$

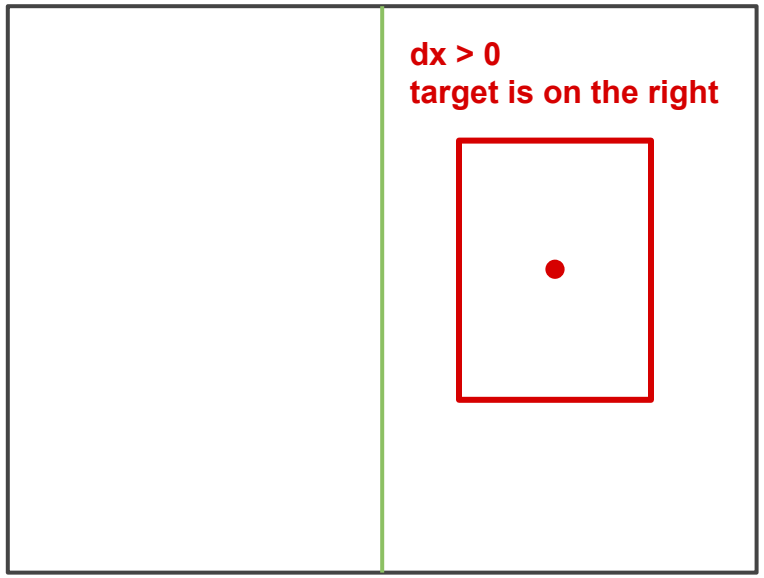
3 · CAMERA FOCAL LENGTH IN PIXELS

$$f = (W \div 2) \div \tan(HFOV \div 2)$$

4 · TURN ANGLE

$$\theta = \operatorname{atan}(dx \div f)$$

Use your camera's real HFOV from its data sheet. This assumes the camera looks along the arm's x axis.



The base angle θ_B counts counter-clockwise from above, so a target on the **right** means a **negative** turn.

DAY 2 · WORKED EXAMPLE

Turn Angle Example

Image width W = **640 px**. Camera HFOV = **60°**.

Detected cup box = **(400, 150, 500, 350)**.

By how much should the base turn?

SOLUTION

cx = (400 + 500) ÷ 2 = 450

dx = 450 − 320 = 130 px

f = 320 ÷ tan(30°) = 554.3 px

θ = atan(130 ÷ 554.3) = 13.2°

The target is to the right, so $\theta_B = -13.2°$

DAY 2 · TRY IT YOURSELF

Try It Yourself #5

Same camera: W = **640 px**, HFOV = **60°**.

This time the box is **(60, 100, 180, 300)**.

Find the base angle θ_B.

SOLUTION

cx = (60 + 180) ÷ 2 = 120

dx = 120 − 320 = −200 px

θ = atan(200 ÷ 554.3) = 19.8°

The target is to the left, so θ_B = +19.8°

DAY 2 · CHECK WITH CODE

The Whole Pipeline in Code

PYTHON

```python
from detect_helper import *

dets = run_detector("table.png", conf=0.10)    # 1-2: photo and boxes
dets = filter_by_score(dets, 0.50)             # 3: drop low scores
dets = nms(dets, 0.5)                          # 3: remove duplicates
target = pick_target(dets, "cup")              # 4: the best cup
dx, dy = pixel_offset(target["box"], 640, 480)
theta_b = -turn_angle(dx, 640, 60)             # 5: base angle
```

CHECK

Feed in the worked example boxes. You should get θ_B = −13.2° and +19.8°.

SIGN CHECK

Right of center must give a negative θ_B. Left must give positive.

WRAP-UP

What We Can Now Do

DAY 1

Read a bounding box and measure overlap with IoU

DAY 1

Run a pretrained detector and read its output

DAY 2

Filter by confidence and remove duplicates with NMS

DAY 2

Score with precision and recall, and turn the base toward a target

LIMIT TO REMEMBER

One camera gives direction but not distance. Next lesson: stereo vision adds depth.